Folha de Respostas da Atividade Aula FSM

Nome: Felipe Uehara Gondo **N.USP: 13680612**

Instruções: Completar com apenas as informações e dados solicitados, e apenas nos campos adequados

As imagens subidas devem se denominadas com a enumeração dos itens (os mesmos da apostila prática).

1) Em relação aos itens de 1.f, faça o upload da imagem da **simulação da FSM** ilustrando o <u>protocolo, um handshake completo dos sinais junto ao estado INIT_ACTIVATION (sinais entre as FSMs)</u>.

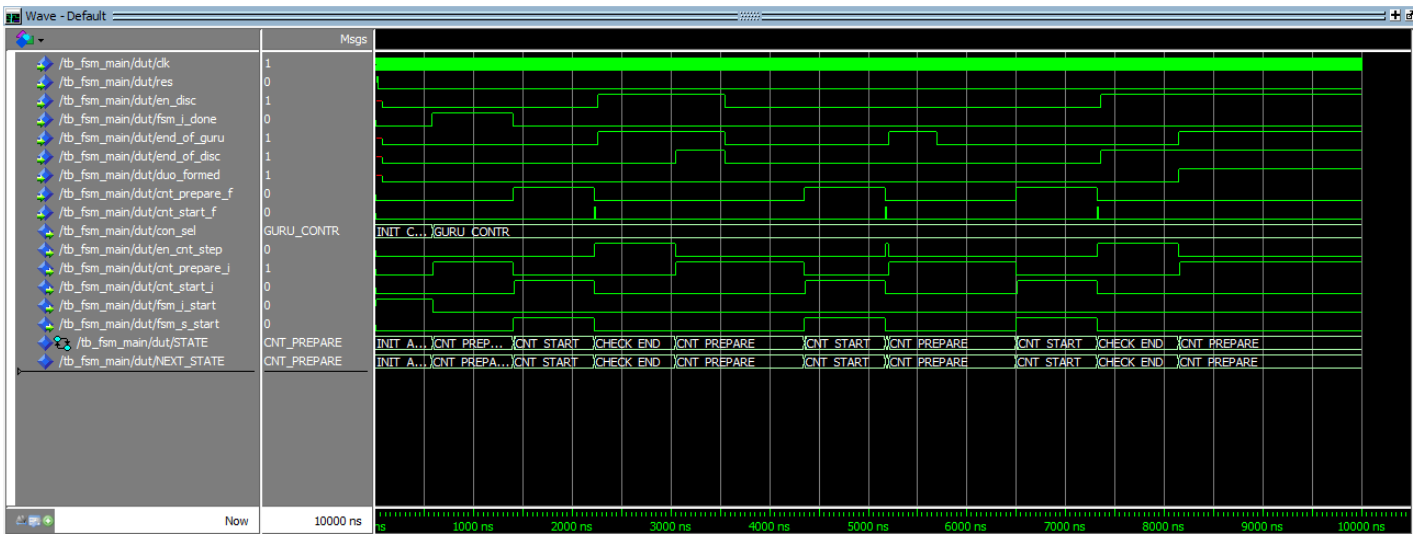


Figura 1: Simulação Completa do testbench tb_fsm_main.vhd

Explique o protocolo da imagem acima, indicando a sequência dos eventos, com os tempos e os valores dos sinais envolvidos na sequência da sinalização.

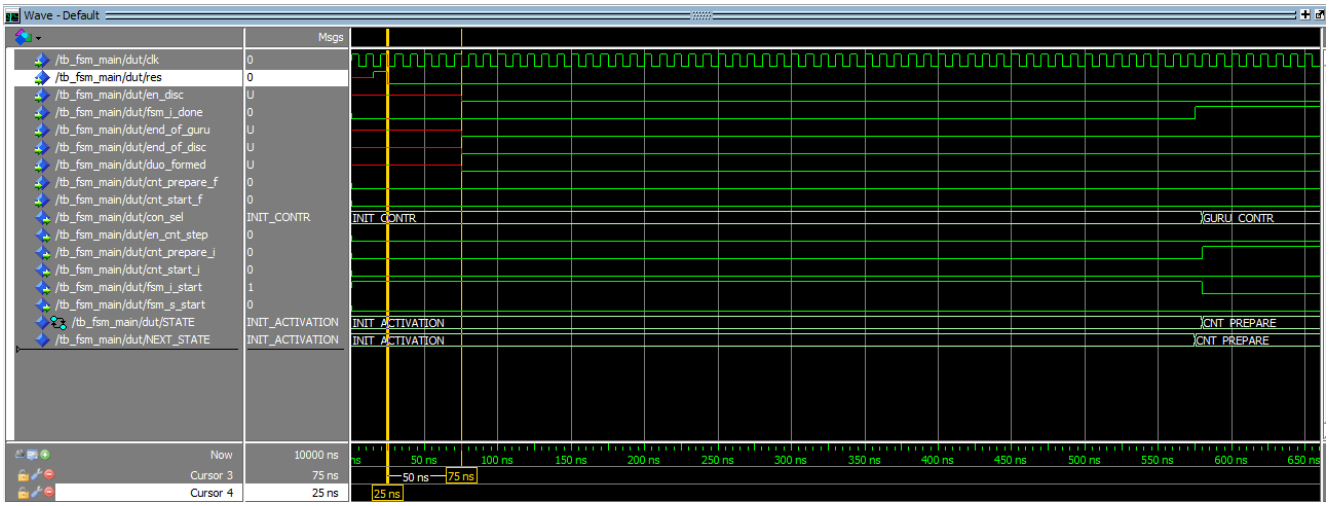


Figura 2: Estado INIT_ACTIVATION do tesbench

Inicialmente, é dado um reset no circuito para eliminar quaisquer valores antigos presentes na memória. Após a borda de descida do sinal de reset, são contados 50ns para definir os valores de enable, do guru, do discípulo e da dupla em si. Com isso, ocorre o estado INIT_ACTIVATION para definir os estados iniciais das variáveis e sinais intermediários. Até o final deste estado, passaram-se 575ns.

1) Ainda em relação aos itens de 1.f, faça o upload da imagem da **simulação da FSM** ilustrando o de protocolo, um handshake completo dos sinais junto ao estado CNT_PREPARE.

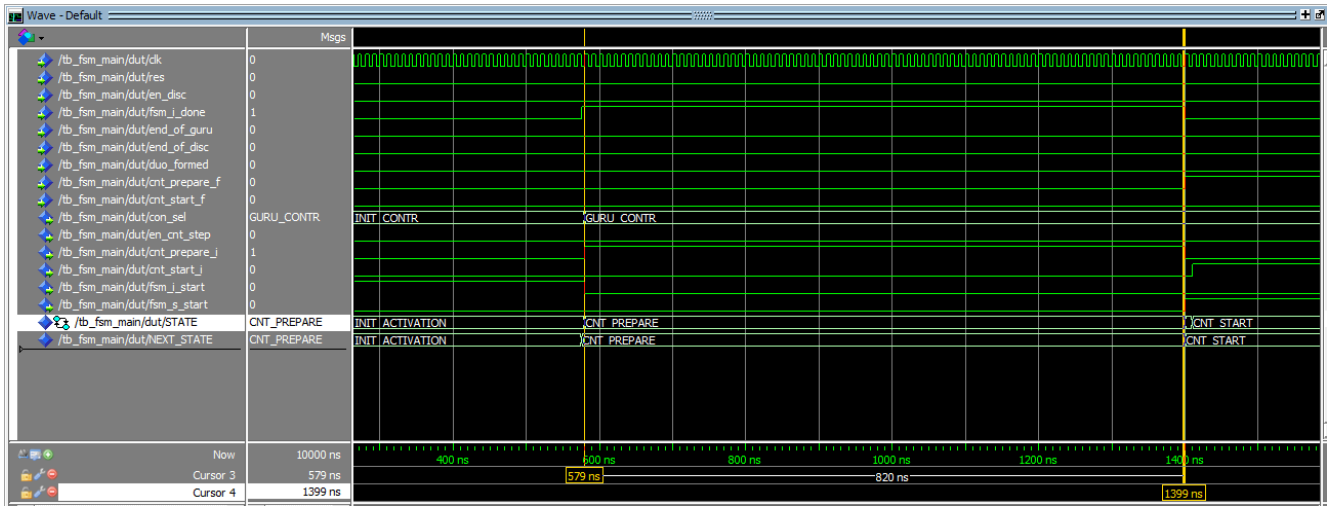


Figura 3: Estado CNT_PREPARE *do tesbench*

No quadro a seguir, explique o protocolo da imagem acima, indicando a sequência dos eventos, com os tempos e os valores dos sinais envolvidos a sequência da sinalização.

Para o estado mudar de INIT_ACTIVATION para CNT_PREPARE, a entrada fsm_i_done mudou seu valor para '1'. Com isso, inicia-se uma contagem de tempo antes de iniciar a corrida do guru e do discípulo em si, tempo este que dura 5ns até a passagem para o estado seguinte. Com isso, passaram-se 580ns da simulação.

1) **k)** Em relação à presença dos estados no Wave, faça o upload da imagem da simulação onde apareça a sequência CNT_PREPARE, STEP_ACTIVATION e CNT_START.

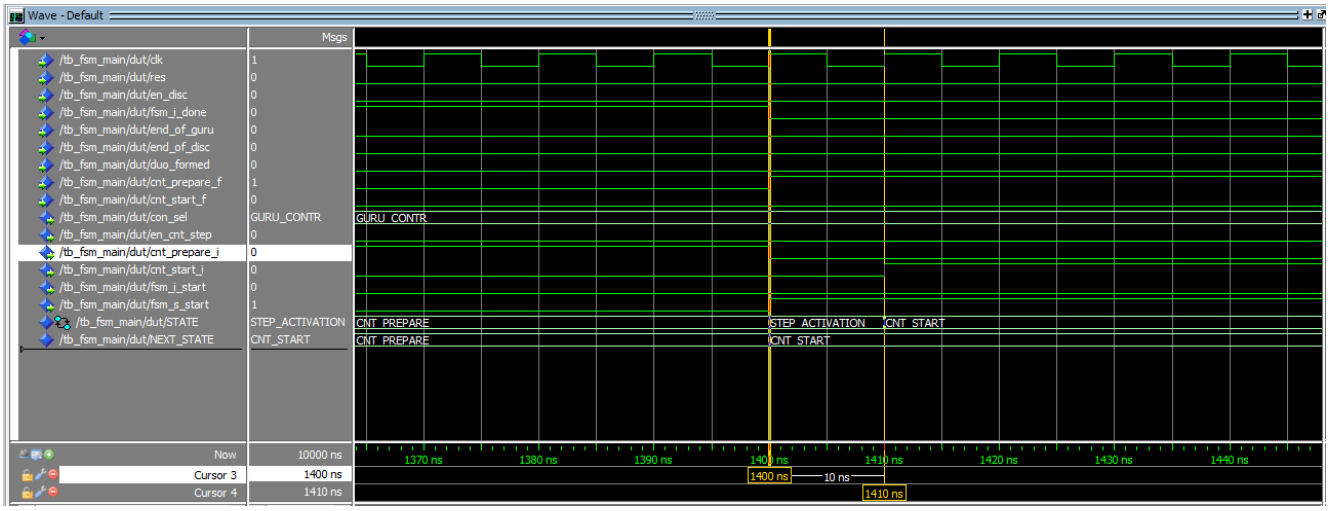


Figura 4: Estado STEP_ACTIVATION *do tesbench*

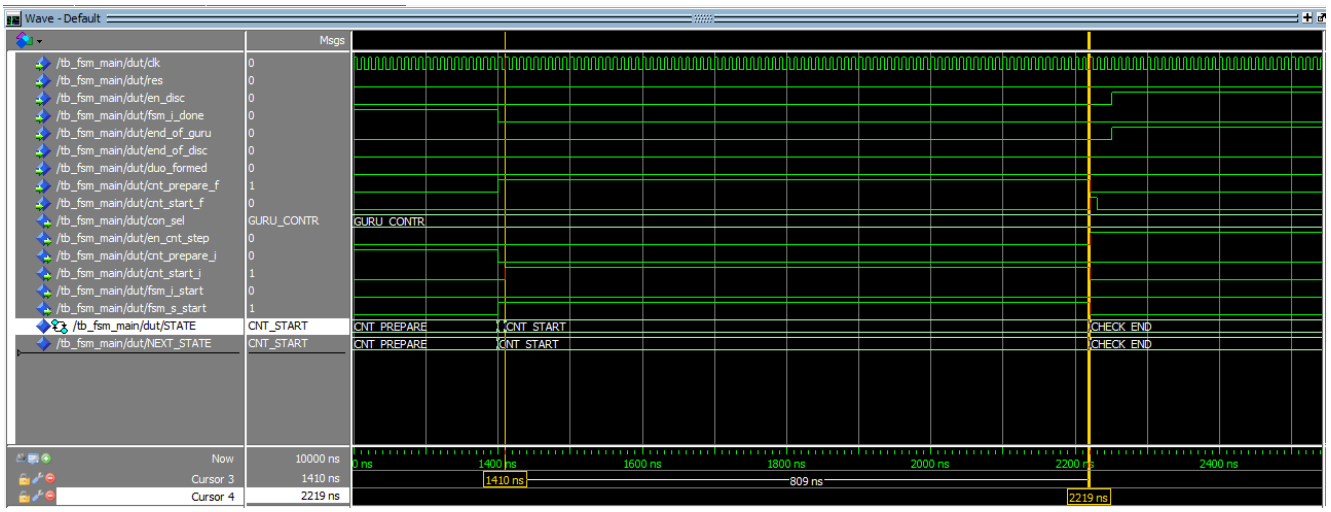


Figura 5: Estado STEP_ACTIVATION e CNT_START *do tesbench*

Até este estado, passaram-se 2.22us desde o início da simulação.

1) **I)** Insira no quadro abaixo o novo processo dentro do módulo stimuli_module, que inclui o teste das condições indicadas no item 1.I.

```
--  ================================
--   ====================================

        -- **step** protocol 2nd time *********

    -- prepare wait protocol

        activate_status('0','0','0','0');    -- no end status again

        while (cnt_prepare_i /= '1')
            loop
            activate_protocol('1','0','0');    -- guaranteeing cnt_prepare_f='0' and
maintaining fsm_i_done='1'
        end loop;


    -- here fsm_main indicates preparation phase should start;  testbench emulates
an expected delay to starr a new round - a shorter version is adopted- only 80
cicles...

        wait for 112*CLK_PERIOD;
        reset_activate;--reset after 112 clock periods

        activate_protocol('0','1','0');         ---- testbench emulates end of delay
with cnt_prepare_f='1'  and forcing fsm_main to go to STEP_ACTIVATION
```

```
        -- forcing cnt_prepare_f='1'

        while (cnt_prepare_i /= '0')
            loop
            activate_protocol('0','1','0');    -- verifying cnt_prepare_i returned
to '0' and maintaining cnt_prepare_f='1'
        end loop;


        activate_protocol('0','1','0');     -- fsm_main indicates the
STEP_ACTIVATION has started
                                            -- still cnt_prepare_f='1'
```

Faça o upload da imagem da simulação correspondente aos estímulos do item anterior. Mostre na figura que o número de ciclos entre o início de CNT_PREPARE e a ativação do reset corresponde ao desejado.

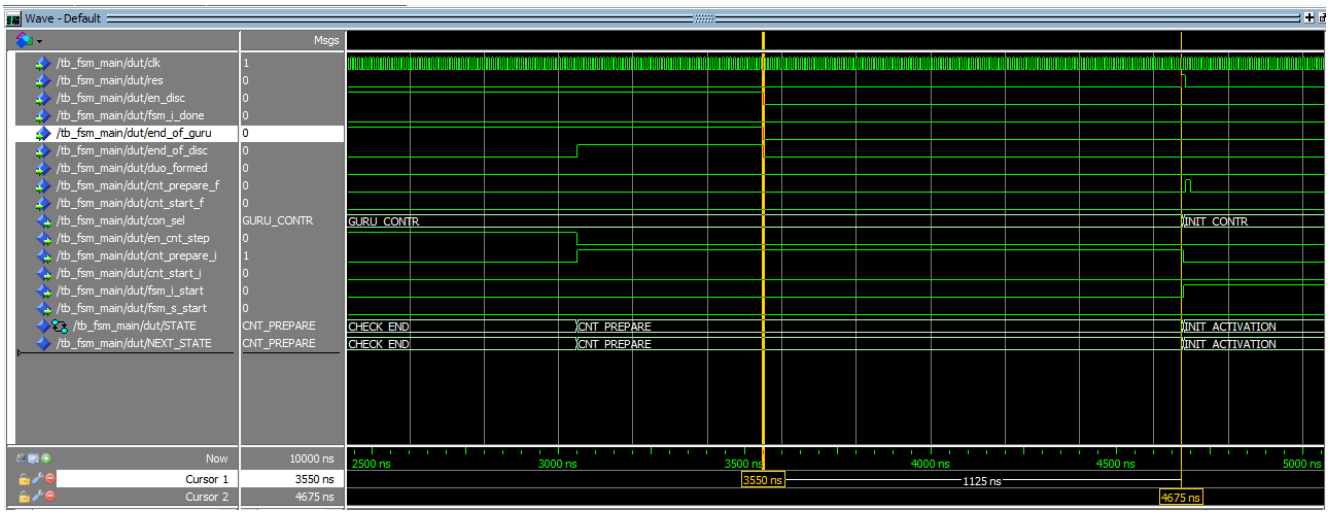


Figura 6: Estado CNT_PREPARE alterado

2) Do projeto VHD resultante :

A) Entity do **fsm_init** (copiar para o quadro):

```
entity fsm_init is
    port
    (
    clk             : in STD_LOGIC;         --from system
    res             : in STD_LOGIC;         --from system
    start_init      : in STD_LOGIC;         --from fsm internal
    ovf_of_y        : in STD_LOGIC;         --from system
    done_init       : out STD_LOGIC;        --to fsm internal
```

```
    flags_2_dp      : out ctrl_2_dp_flags;  --to datapath
    flags_2_mem     : out base_2_mem_flags  --to memory
    );
end fsm_init;
```

B) Arquitetura do fsm_init:

Declaração de Tipos e Sinais (copiar para o quadro).

```
ARCHITECTURE ARCH OF fsm_init IS
    TYPE state_type IS (ST_IDLE, ST_INIT_LOOP, ST_DONE);  -- veja quantos estados
hah no seu projeto
    SIGNAL state, next_state : state_type;

BEGIN
    ------------------------Lógica Sequencial-----------------------
    SEQ: PROCESS (res, clk)
    BEGIN
        IF (res='1') THEN
            state <= ST_IDLE;
        ELSIF Rising_Edge(clk) THEN
            state <= next_state;
        END IF;
    END PROCESS SEQ;
    -----------------------Lógica Combinacional do estado siguinte--
    COMB: PROCESS (start_init, ovf_of_y, state)  -- completar com sinais de entrada
+ state
    BEGIN
        CASE state IS
    -- para cada estado, preencha as condicoes para transicao de estado
            WHEN ST_IDLE =>
                IF (start_init = '1') THEN
                    next_state <= ST_INIT_LOOP;
                ELSE
                    next_state <= ST_IDLE;
                END IF;

                WHEN ST_DONE =>
                -- Finaliza e retorna ao estado ocioso (ou aguarda start_init
descer)
                IF (start_init = '0') THEN
                    next_state <= ST_IDLE;
                ELSE
```

```
                next_state <= ST_DONE;
            END IF;

            WHEN OTHERS =>
            next_state <= ST_IDLE;
    END CASE;
END PROCESS COMB;

-----------------------Lógica Combinacional saidas--------------------
-----------------------Lógica Combinacional de Saídas--------------------
SAI: PROCESS (state)
BEGIN
    -- 1. Atribuições padrão (Evita a criação de latches indesejados)
    done_init                 <= '0';

    flags_2_dp.ng_rand_zero   <= '0';
    flags_2_dp.ng_cte_incr    <= '0';
    flags_2_dp.rb_init_en     <= '0';
    flags_2_dp.rb_out_sel     <= INIT_OUT;
    flags_2_dp.alu_ctrl       <= '0';
    flags_2_dp.cg_sel         <= BLANK;

    flags_2_mem.mem_wr_en     <= '0';
    flags_2_mem.cg_sel        <= BLANK;

    -- 2. Atribuições específicas dependentes APENAS do estado atual (Moore)
    CASE state IS
        WHEN ST_IDLE =>
            -- Nenhuma saída ativa, mantêm-se os valores padrão

        WHEN ST_INIT_LOOP =>
            -- Habilita escrita na memória passando o código "BLANK"
            flags_2_mem.mem_wr_en     <= '1';
            flags_2_mem.cg_sel        <= BLANK;

            -- Sinais para o datapath (incrementar contadores, habilitar inits)
            flags_2_dp.rb_init_en     <= '1';
            flags_2_dp.ng_cte_incr    <= '1';
            flags_2_dp.rb_out_sel     <= INIT_OUT;
            flags_2_dp.cg_sel         <= BLANK;

        WHEN ST_DONE =>
            -- Sinaliza que o processo de inicialização terminou
            done_init                 <= '1';
```

```
            WHEN OTHERS =>
                -- Valores padrão são mantidos
        END CASE;
    END PROCESS SAI;

END ARCH;
```

Designações de Sincronismo de Estado (bloco SEQ) (copiar para o quadro):

```
-----------------------Lógica Sequencial----------------------
    SEQ: PROCESS (res, clk)
    BEGIN
        IF (res='1') THEN
            state <= ST_IDLE;
        ELSIF Rising_Edge(clk) THEN
            state <= next_state;
        END IF;
    END PROCESS SEQ;
```

Designações da Lógica do Estado Futuro (bloco COMB) (copiar para o quadro):

```
-----------------------Lógica Combinacional do estado siguinte--
    COMB: PROCESS (start_init, ovf_of_y, state)  -- completar com sinais de entrada
+ state
    BEGIN
        CASE state IS
    -- para cada estado, preencha as condicoes para transicao de estado
            WHEN ST_IDLE =>
                IF (start_init = '1') THEN
                    next_state <= ST_INIT_LOOP;
                ELSE
                    next_state <= ST_IDLE;
                END IF;

                WHEN ST_DONE =>
                -- Finaliza e retorna ao estado ocioso (ou aguarda start_init
descer)
                IF (start_init = '0') THEN
                    next_state <= ST_IDLE;
                ELSE
```

```
                    next_state <= ST_DONE;
                END IF;

                WHEN OTHERS =>
                next_state <= ST_IDLE;
        END CASE;
    END PROCESS COMB;
```

Designações da Lógica de Saída (bloco SAI) (copiar para o quadro):

```
-----------------------Lógica Combinacional de Saídas--------------------
    SAI: PROCESS (state)
    BEGIN
        -- 1. Atribuições padrão (Evita a criação de latches indesejados)
        done_init                   <= '0';

        flags_2_dp.ng_rand_zero     <= '0';
        flags_2_dp.ng_cte_incr      <= '0';
        flags_2_dp.rb_init_en       <= '0';
        flags_2_dp.rb_out_sel       <= INIT_OUT;
        flags_2_dp.alu_ctrl         <= '0';
        flags_2_dp.cg_sel           <= BLANK;

        flags_2_mem.mem_wr_en       <= '0';
        flags_2_mem.cg_sel          <= BLANK;

        -- 2. Atribuições específicas dependentes APENAS do estado atual (Moore)
        CASE state IS
            WHEN ST_IDLE =>
                -- Nenhuma saída ativa, mantêm-se os valores padrão

            WHEN ST_INIT_LOOP =>
                -- Habilita escrita na memória passando o código "BLANK"
                flags_2_mem.mem_wr_en       <= '1';
                flags_2_mem.cg_sel          <= BLANK;

                -- Sinais para o datapath (incrementar contadores, habilitar inits)
                flags_2_dp.rb_init_en       <= '1';
                flags_2_dp.ng_cte_incr      <= '1';
                flags_2_dp.rb_out_sel       <= INIT_OUT;
                flags_2_dp.cg_sel           <= BLANK;

            WHEN ST_DONE =>
                -- Sinaliza que o processo de inicialização terminou
```

```
                done_init                <= '1';

            WHEN OTHERS =>
                -- Valores padrão são mantidos
        END CASE;
    END PROCESS SAI;
```

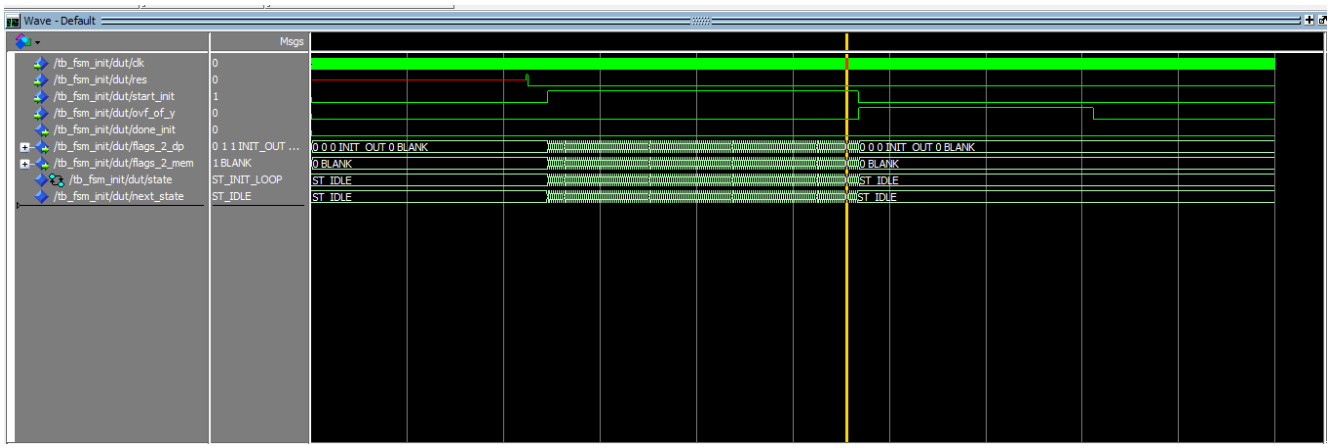
3) k) Insira no quadro abaixo o **trecho do processo** dentro do módulo **stimuli_fsm_init**, que inclui o teste das condições indicadas no item 3.k.

```
procedure assign_condition(start_init_value, ovf_of_y_value : in STD_LOGIC) is
        begin
            -- Assign values to estimuli_module´s outputs.
            start_init <= start_init_value;
            ovf_of_y <= ovf_of_y_value;

            wait for 111*CLK_PERIOD;
            wait until rising_edge (clk_s);
            -- Events at the rising edge of next clock cycle
            end procedure assign_condition;

        procedure reset_activate is    -- reset activation procedure
        begin
            wait until falling_edge(CLK_s);
            rst <= '1';
            wait for CLK_PERIOD;
            rst <= '0';
        end procedure reset_activate;
```

Suba a imagem da simulação <u>correspondente aos estímulos deste item</u>.



Explique como pode-se verificar na imagem acima que as condições definidas para 3.k foram atendidas (aponte os tempos de inicialização e finalização da escrita)